

1. Objetivo

Este documento define a estratégia de testes para o sistema LEVAAÍ, focando na validação das regras de negócio (Domain-Driven Design), na resiliência da arquitetura orientada a eventos e na integração correta entre os microcomponentes (API Principal, Message Broker e Worker). O objetivo é garantir uma cobertura de código confiável e assegurar que falhas parciais não resultem em perda de dados dos clientes.

2. Níveis de Teste Aplicados

- **Testes Unitários:** Focados no núcleo da aplicação (Regras de Negócio/Domínio), garantindo que os cálculos e lógicas funcionem de forma isolada, sem dependência de banco de dados ou mensageria.
- **Testes de Integração:** Focados em validar a comunicação do sistema com componentes externos (RabbitMQ, Banco de Dados Relacional e Google Maps API).
- **Testes de Resiliência (Caos Simulado):** Cenários que validam o comportamento do sistema diante de falhas de rede ou indisponibilidade de serviços.

3. Cenários de Teste Críticos (Casos de Teste)

Categoria A: Regras de Negócio e Matching (Testes Unitários)

- **CT-A01:** Validar se o sistema rejeita um transportador cujo veículo (ex: Picape) possui capacidade de peso inferior à informada no frete pelo contratante.
- **CT-A02:** Validar se o sistema rejeita um transportador cujo veículo possui capacidade de volume (m³) inferior à informada no frete.
- **CT-A03:** Garantir que o cálculo do valor base do frete aplica a estratégia (Strategy Pattern) correta de acordo com a categoria do veículo (Picape vs. Caminhão).

Categoria B: Fluxo Assíncrono e Mensageria (Testes de Integração)

- **CT-B01:** Verificar se a *API Principal*, ao receber uma solicitação de orçamento válida, retorna imediatamente o status 202 Accepted para o Front-end e grava o status "Processando" no banco.
- **CT-B02:** Validar se a *API Principal* publica com sucesso o evento PedidoCriado na fila correta do RabbitMQ, contendo o *payload* em formato JSON.
- **CT-B03:** Assegurar que o *Worker de Matching* consome a mensagem da fila do RabbitMQ e altera o status do pedido no banco de dados para "Buscando Transportador".

Categoria C: Resiliência e Tolerância a Falhas (Sistemas Distribuídos)

- **CT-C01 (Queda do Worker):** Simular o desligamento do *Worker*. A API deve continuar recebendo pedidos normalmente, e as mensagens devem ficar retidas (acumuladas) na fila do RabbitMQ de forma segura até o *Worker* ser reiniciado.

- **CT-C02 (Falha na API Externa):** Simular indisponibilidade temporária (Timeout) na API do Google Maps. O *Worker* deve capturar a exceção e devolver a mensagem para a fila (DLQ - Dead Letter Queue) ou tentar o reprocessamento (Retry) após alguns segundos, sem perder o pedido do cliente.

Categoria D: Notificações em Tempo Real (WebSockets)

- **CT-D01:** Verificar se o sistema dispara com sucesso um evento via *WebSocket* contendo a lista de motoristas assim que o *Worker* finaliza o *matching*.
- **CT-D02:** Validar se o Front-end atualiza a lista de transportadores na tela do usuário instantaneamente ao receber a mensagem pelo canal do *WebSocket*, sem necessidade de recarregar a página (F5).